

Chapter 01: Python Basics and Particle Projectile Simulation

From Fundamentals to Plasma Physics Applications

Pakorn Wongwaitayakornkul, PhD

PHY653: Computational Electromagnetics and Plasma Physics

- 1 Motivation
- 2 Plasma Physics Fundamentals
- 3 Python Basics
- 4 Projectile Motion Simulation
- 5 Example: Parabola of Safety
- 6 Example: Rutherford Scattering
- 7 Summary

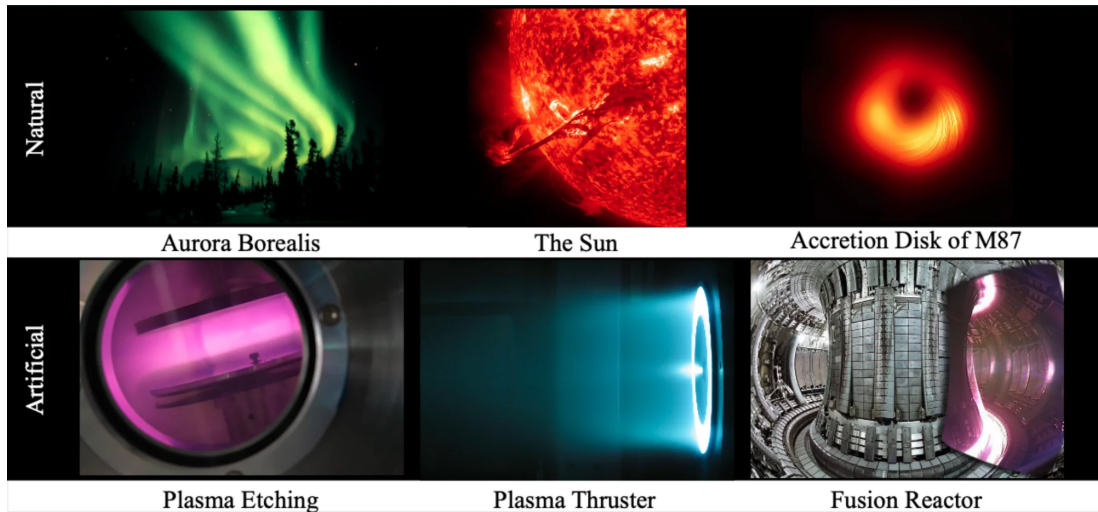
Why Python for Physics?

Python is ideal for scientific computing:

- High-level language that is easy to learn and use
- Open-source with a large community of users and developers
- Extensive libraries for numerical computing (NumPy, SciPy)
- Powerful visualization tools (Matplotlib)
- Jupyter notebooks combine code, text, $\text{\LaTeX}$ equations, and visualizations

Our Goal: Create a particle projectile simulation to study the motion of charged particles — a fundamental problem in plasma physics.

Plasma: The Fourth State of Matter



Plasma appears in nature (aurora, sun, accretion disks) and in technology (etching, thrusters, fusion reactors).

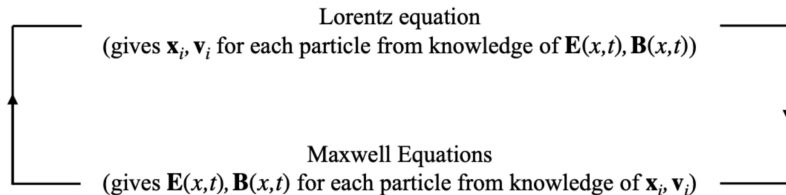
The study of plasma physics is built on the **self-consistent interaction** between:

- **Electromagnetic fields** — influenced by charged particle motion
- **Charged particles** — influenced by electromagnetic fields

This interplay leads to complex collective behaviors:

- Plasma oscillations
- Wave-particle interactions
- Instabilities

The Fundamental Coupling



The **Lorentz equation** governs particle motion given the fields, while **Maxwell's equations** determine the fields from particle distributions.

The Lorentz Equation

The motion of a charged particle in electromagnetic fields:

$$m \frac{d\mathbf{v}}{dt} = q(\mathbf{E} + \mathbf{v} \times \mathbf{B})$$

Variables:

- m — mass of the particle
- q — charge of the particle
- $\mathbf{v}$ — velocity of the particle
- $\mathbf{E}$ — electric field
- $\mathbf{B}$ — magnetic field

Maxwell's Equations

The behavior of electromagnetic fields in space and time:

Gauss's Law (Electric):

$$\nabla \cdot \mathbf{E} = \frac{\rho}{\varepsilon_0}$$

Faraday's Law:

$$\nabla \times \mathbf{E} = -\frac{\partial \mathbf{B}}{\partial t}$$

Gauss's Law (Magnetic):

$$\nabla \cdot \mathbf{B} = 0$$

Ampère-Maxwell Law:

$$\nabla \times \mathbf{B} = \mu_0 \mathbf{J} + \mu_0 \varepsilon_0 \frac{\partial \mathbf{E}}{\partial t}$$

Here ρ is charge density, $\mathbf{J}$ is current density, ε_0 is vacuum permittivity, and μ_0 is vacuum permeability.

Essential libraries for scientific computing:

```
# Import necessary libraries
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation
```

- `numpy` — numerical arrays and mathematical operations
- `matplotlib.pyplot` — plotting and visualization
- `FuncAnimation` — creating animations

Creating arrays and plotting a sinusoidal function:

```
# Create an array of x values
x = np.linspace(0, 2*np.pi, 100)

# Compute sine values
y = np.sin(x)

# Create the plot
plt.figure(figsize=(8, 4))
plt.plot(x, y, 'b-', linewidth=2)
plt.xlabel('x')
plt.ylabel('sin(x)')
plt.title('Sinusoidal Function')
plt.grid(True)
plt.show()
```

Equation of Motion

Vertical motion under gravity:

$$y(t) = y_0 + v_{y,0}t - \frac{1}{2}gt^2$$

Horizontal motion (no forces):

$$x(t) = x_0 + v_{x,0}t$$

Variables:

- y_0, x_0 — initial position
- $v_{y,0}, v_{x,0}$ — initial velocity components
- g — gravitational acceleration

Analogy: Gravity and Electric Field

Gravitational acceleration:

$$y(t) = y_0 + v_{y,0}t - \frac{1}{2}gt^2$$

Charged particle in constant electric field:

$$y(t) = y_0 + v_{y,0}t + \frac{1}{2} \frac{qE}{m} t^2$$

Key insight: The constant gravitational acceleration g is analogous to a constant electric field E acting on a charged particle. Both produce uniform acceleration!

Modeling inelastic collisions:

When the particle bounces off the ground, it loses energy. We model this by:

$$v_{\text{new}} = -v_{\text{old}} \times (\text{energy loss fraction})$$

$$y_{\text{new}} = -y_{\text{old}} \times (\text{energy loss fraction})$$

A typical energy loss fraction is 0.8 (20% energy loss per bounce).

Simulation Parameters

```
# Parameters
initial_height = 10.0      # meters
initial_velocity = 5.0     # m/s
initial_distance = 0.0    # meters
initial_angle = 45.0      # degrees
energy_loss_fraction = 0.8
gravity = 9.81            # m/s^2

# Time parameters
dt = 0.1      # time step (seconds)
tmax = 10     # maximum time (seconds)
```

Simulation Loop Structure

```
# Initialize arrays
time = np.arange(0, tmax, dt)
x = np.zeros(len(time))
y = np.zeros(len(time))
vx = np.zeros(len(time))
vy = np.zeros(len(time))

# Set initial conditions
x[0], y[0] = initial_distance, initial_height
vx[0] = initial_velocity * np.cos(np.radians(angle))
vy[0] = initial_velocity * np.sin(np.radians(angle))

# Main loop
for i in range(1, len(time)):
    # Update velocity and position
    # Check for bounce condition
```

The Parabola of Safety

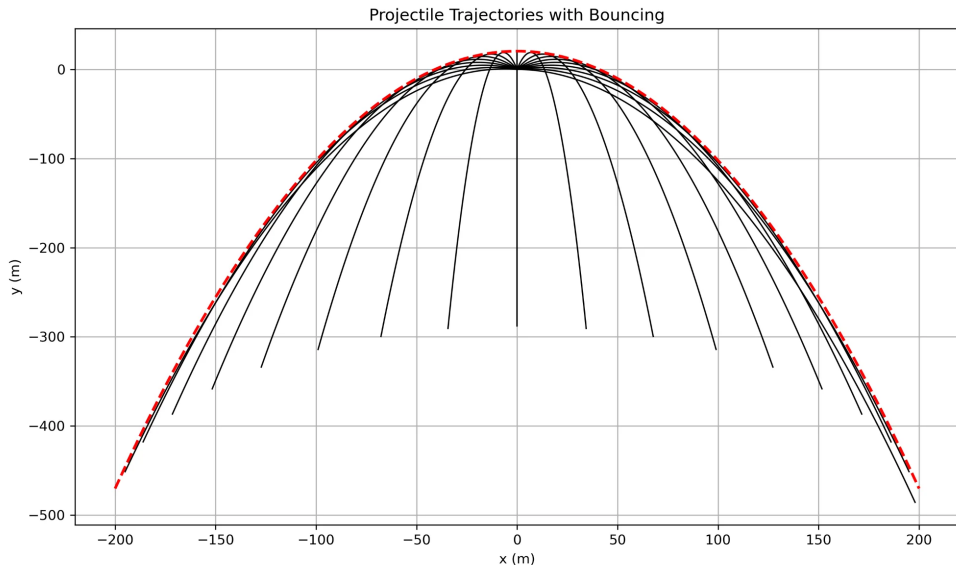
Problem: A projectile launched from height h with initial velocity u at angle θ . What is the envelope of all possible trajectories?

The parabola of safety:

$$y = \frac{u^2}{2g} - \frac{gx^2}{2u^2}$$

This curve represents the maximum height reachable at each horizontal distance for *any* launch angle.

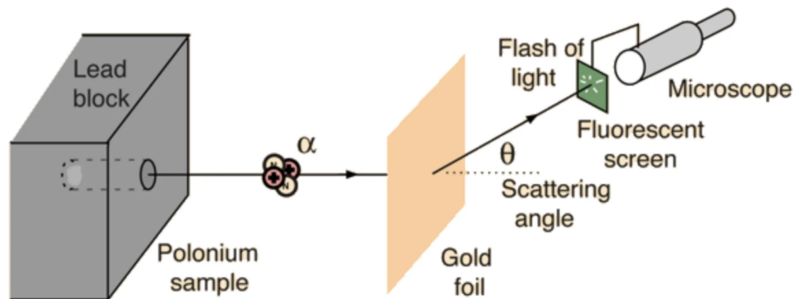
Trajectories and Safety Envelope



Multiple trajectories at different angles. The dashed red curve is the parabola of safety — no projectile can reach beyond this envelope.

Rutherford Scattering: Non-Constant Electric Field

Setup: Alpha particles scattered by a gold nucleus.



The electric field is **not constant** — it follows the Coulomb law:

$$\vec{E} = \frac{q_2}{4\pi\epsilon_0 r^2} \hat{r}$$

Scattering Angle Formula

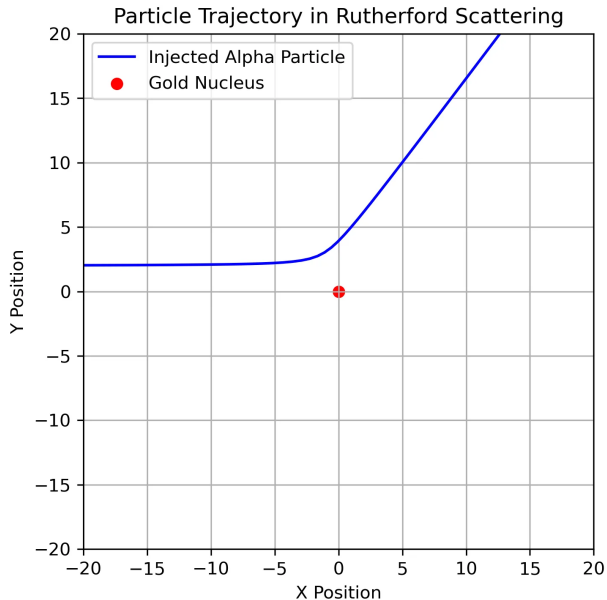
The Rutherford scattering formula:

$$\tan\left(\frac{\theta}{2}\right) = \frac{q_1 q_2}{4\pi\epsilon_0 \mu b v_\infty^2}$$

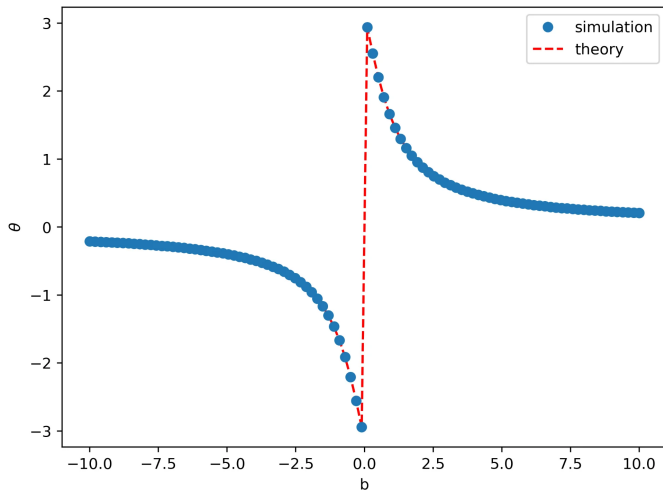
Variables:

- θ — scattering angle
- b — impact parameter (perpendicular distance from nucleus)
- v_∞ — initial velocity at infinity
- μ — reduced mass
- q_1, q_2 — charges of alpha particle and nucleus

Simulated Trajectory



Verification: Impact Parameter vs. Scattering Angle



The simulation (blue dots) matches the theoretical prediction (red dashed line) for the relationship between impact parameter b and scattering angle θ .

What we covered:

- ① **Python basics:** Arrays, loops, plotting, and animations
- ② **Plasma physics foundations:** Lorentz equation and Maxwell's equations
- ③ **Projectile motion:** Equations of motion with energy loss
- ④ **Analogy:** Gravity $\leftrightarrow$ constant electric field
- ⑤ **Parabola of safety:** Envelope of all trajectories
- ⑥ **Rutherford scattering:** Non-constant electric field simulation

Next steps: Extend these simulations to include magnetic fields and study charged particle dynamics in plasmas!

Key Equations Summary

Lorentz Force:

$$m \frac{d\mathbf{v}}{dt} = q(\mathbf{E} + \mathbf{v} \times \mathbf{B})$$

Projectile Motion:

$$y(t) = y_0 + v_{y,0}t - \frac{1}{2}gt^2$$

Parabola of Safety:

$$y = \frac{u^2}{2g} - \frac{gx^2}{2u^2}$$

Rutherford Scattering:

$$\tan\left(\frac{\theta}{2}\right) = \frac{q_1 q_2}{4\pi\epsilon_0 \mu b v_\infty^2}$$